

Final Project Submission Template

Page 1 – Cover Page

<i>Student name</i>	Ilie Beleniuc
<i>Student number</i>	260417001
<i>Lesson name and number</i>	Internet and Web Programming – SEN204

Page 2 – Introduction

Project Title:

To-Do List v2

Research Question / Problem Statement:

How can a simple personal to-do list application be extended beyond single-device browser storage so that tasks persist across sessions and devices without becoming overly complex?

The mid-term version of this project stored tasks in the browser's `localStorage`. This meant that tasks were lost whenever the browser data was cleared and could not be accessed from a different device or browser. The goal of version 2 is to solve this by moving task storage to a real database on a backend server while keeping the user interface just as simple.

Objective of the Project:

The project aims to build a fullstack web application that lets a user manage a personal task list through a browser. The frontend is written in React and communicates with a Django REST API over HTTP. Tasks are stored in an SQLite database on the server. The entire application can be started with a single `docker compose up` command, which removes the need to install Python or Node on the host machine.

Expected Outcome:

A working fullstack web application where the user can:

- Add tasks with a title, optional notes, a priority level, and an optional due date.
- Mark tasks as completed or active using a checkbox.
- Edit a task's title inline without leaving the page.
- Delete tasks with a confirmation prompt.
- Filter the list to show All, Active, or Completed tasks.

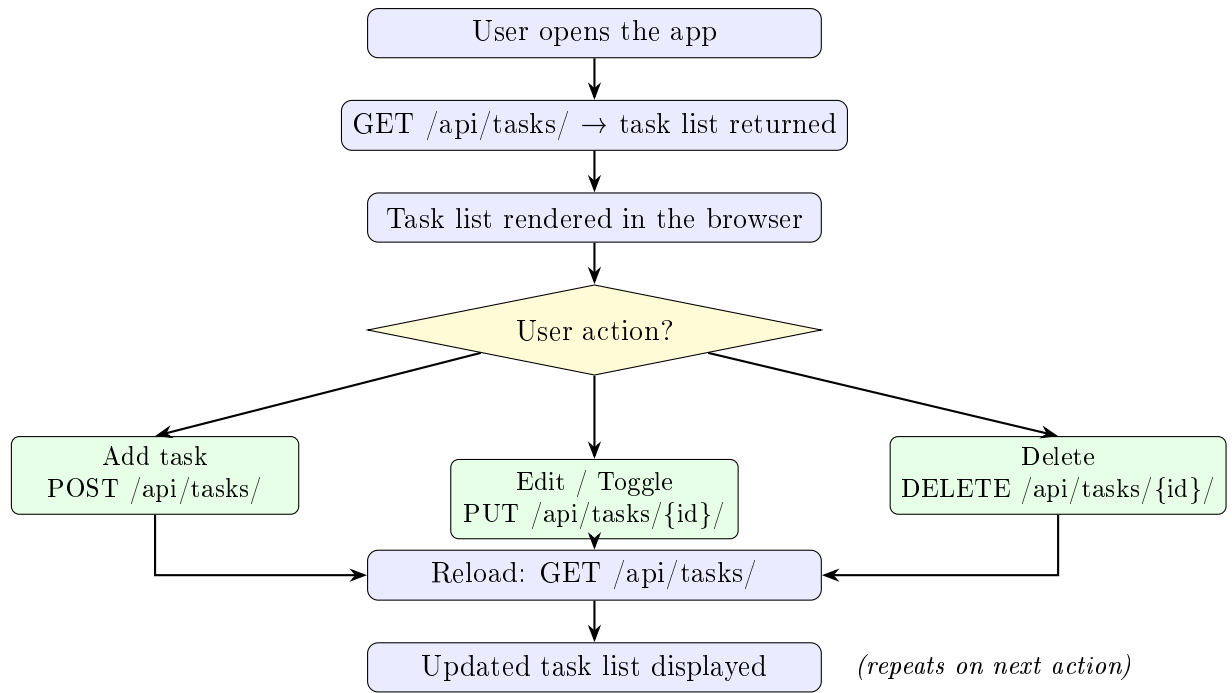
The project demonstrates the React, Django, and Docker topics covered throughout SEN204 and improves meaningfully on the mid-term submission.

Approach to Problem Solving:

- **Input:** User interactions in the browser — typing a task title, clicking Add, checking a checkbox, clicking Edit or Delete, or selecting a filter button.
- **Processing:** The React frontend sends HTTP requests (GET, POST, PUT, DELETE) to the Django REST API. The API validates the data, reads from or writes to the SQLite database, and returns a JSON response.
- **Output:** The UI re-renders to reflect the updated task list.

Control Flow:

1. User opens `http://localhost:5173` in the browser.
2. The Dashboard component mounts and calls `GET /api/tasks/`.
3. The API queries the database and returns the full task list as JSON.
4. The task list is rendered. For each task: a checkbox, the title (or an inline input when editing), the priority badge, the due date, and Edit/Delete buttons are shown.
5. **Add task:** User fills the form and clicks *Add Task* → `POST /api/tasks/` → task saved → list reloaded.
6. **Toggle complete:** User clicks the checkbox → `PUT /api/tasks/{id}/` with `completed: true/false` → list reloaded.
7. **Edit title:** User clicks *Edit* → inline input appears → user presses Enter or clicks *Save* → `PUT /api/tasks/{id}/` → list reloaded.
8. **Delete task:** User clicks *Delete* → browser confirm dialog → `DELETE /api/tasks/{id}/` → list reloaded.
9. **Filter:** User clicks All / Active / Completed → `GET /api/tasks/` with `?completed=` parameter → list reloaded with filtered results.



Page 4 – Technical Implementation

Programming Languages: Python 3.12 (Django backend), JavaScript (React frontend)

Code Example 1 – Django view for the task list (backend/api/views.py):

```
1 from rest_framework import status
2 from rest_framework.decorators import api_view
3 from rest_framework.response import Response
4 from .models import Task
5 from .serializers import TaskSerializer
6
7 @api_view(['GET', 'POST'])
8 def task_list(request):
9     if request.method == 'GET':
10         tasks = Task.objects.all().order_by('completed', '-created_at')
11         completed = request.query_params.get('completed')
12         if completed == 'true':
13             tasks = tasks.filter(completed=True)
14         elif completed == 'false':
15             tasks = tasks.filter(completed=False)
16         return Response(TaskSerializer(tasks, many=True).data)
17
18     # POST create a new task
19     serializer = TaskSerializer(data=request.data)
20     if not serializer.is_valid():
21         return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)
22     serializer.save()
23     return Response(serializer.data, status=status.HTTP_201_CREATED)
```

Code Example 2 – React fetch wrapper (frontend/src/api.js):

```
1 const API_URL = import.meta.env.VITE_API_URL || '/api';
2
3 export async function apiRequest(path, options = {}) {
4     const headers = {
5         'Content-Type': 'application/json',
6         ...(options.headers || {}),
7     };
8     const response = await fetch(`${API_URL}${path}`, { ...options, headers });
9     const text = await response.text();
10    const data = text ? JSON.parse(text) : null;
11    if (!response.ok) {
12        const err = new Error('Request failed');
13        err.data = data;
14        err.status = response.status;
15        throw err;
16    }
```

```
17     return data;
18 }
```

Explanation:

task_list view — the `@api_view` decorator from Django REST Framework restricts the endpoint to `GET` and `POST` requests only. On `GET` the view reads the `completed` query parameter to optionally filter results, then serializes the queryset to JSON. On `POST` it validates the incoming JSON through `TaskSerializer` before saving to the database, which prevents invalid data from reaching the model.

apiRequest — instead of repeating `fetch()` calls in every component, all HTTP requests go through this single function. It always sets the `Content-Type` header, parses the response body whether or not the status is OK, and throws an error that carries the server's error data so the component can display a meaningful message to the user.